

```

# =====
# CELL 1.1: FULL AUGMENTED MATRIX GAUSSIAN ELIMINATION (REF & RREF)
# =====

import random

# --- 1. Pure Python Mathematical Helper Functions ---
def custom_abs(x):
    """Returns the absolute value of x without the built-in abs()."""
    return x if x >= 0 else -x

def custom_len(lst):
    """Calculates the length of a list without the built-in len()."""
    count = 0
    for _ in lst:
        count += 1
    return count

def clone_matrix(M, rows, cols):
    """Deep clones a 2D matrix structure using nested loops (no slicing)."""
    cloned = []
    for i in range(rows):
        row = []
        for j in range(cols):
            row.append(M[i][j])
        cloned.append(row)
    return cloned

def generate_random_float():
    """Generates a random float of the form n.dddddddd (exactly 8 decimal places)."""
    integer_part = random.randint(1, 9)
    decimal_part = random.randint(10000000, 99999999)
    return float(f"{integer_part}.{decimal_part}")

# --- 2. Interactive Input and Validation Loop ---
print("---- Interactive Matrix Configuration ----")
while True:
    try:
        m = int(input("Enter number of rows (m) for matrix A: "))
        if m <= 0:
            print("Error: Matrix dimension must be a positive integer.")
            continue
        break
    except ValueError:
        print("Error: Invalid entry. Please enter a positive integer.")

while True:
    try:
        n = int(input(f"Enter number of columns (n) for matrix A (must be strictly greater than {m}): "))
        if n <= 0:
            print("Error: Matrix dimension must be a positive integer.")
            continue
        if m >= n:
            print(f"Error: The syllabus mandates m < n always. Since m = {m}, n must be greater than {m}.")
            continue
        break
    except ValueError:
        print("Error: Invalid entry. Please enter a positive integer.")

while True:
    try:
        vector_size = int(input(f"Enter the size of vector b (must equal m which is {m}): "))
        if vector_size <= 0:
            print("Error: Vector size must be positive.")
            continue
    
```

```

        if vector_size != m:
            print(f"Error: For Ax = b, vector b must align with the rows of A (size must
              continue
        break
    except ValueError:
        print("Error: Invalid entry. Please enter a positive integer.")

# --- 3. Matrix & Vector Population ---
# Automatically generate entries using 8-decimal floats to meet submission requirements
A = []
for _ in range(m):
    row = []
    for _ in range(n):
        row.append(generate_random_float())
    A.append(row)

b = []
for _ in range(m):
    b.append(generate_random_float())

print("\nSuccessfully generated the random linear system!")
print("\n--- Coefficient Matrix A ---")
for row in A:
    print "[" + ", ".join(f"{x:12.8f}" for x in row) + "]"

print("\n--- Vector b ---")
for val in b:
    print(f"[{val:12.8f}]")

# --- 4. Gaussian Elimination Algorithm (REF & RREF) ---
def compute_ref_rref_pure(A, b, m, n):
    """
    Constructs the augmented matrix [A | b] of size m x (n + 1) and computes
    its Row Echelon Form (REF) and Reduced Row Echelon Form (RREF) from scratch.
    """
    # 1. Construct augmented matrix of size m x (n + 1)
    M = []
    for i in range(m):
        row = []
        for j in range(n):
            row.append(A[i][j])
        row.append(b[i]) # Append the vector b entry at index n
        M.append(row)

    ref_M = clone_matrix(M, m, n + 1)
    r = 0
    c = 0
    pivot_rows_cols = [] # Stores pivot coordinate tuples (row, col)

    # 2. Forward elimination to REF
    while r < m and c < n:
        pivot_row = r
        max_val = custom_abs(ref_M[r][c])
        for i in range(r + 1, m):
            val_abs = custom_abs(ref_M[i][c])
            if val_abs > max_val:
                max_val = val_abs
                pivot_row = i

        # Division-by-zero protection:
        if max_val < 1e-9:
            # Skip this column if all entries are zero (marks a free variable)
            c += 1
            continue

        # Row swap if a larger absolute entry was found below
        if pivot_row != r:

```

```

    pivot_row = r
    temp = ref_M[r]
    ref_M[r] = ref_M[pivot_row]
    ref_M[pivot_row] = temp

    # Normalize the pivot row so the leading entry equals 1.0
    pivot_val = ref_M[r][c]
    for j in range(c, n + 1):
        ref_M[r][j] /= pivot_val

    # Eliminate entries in current column for all rows below
    for i in range(r + 1, m):
        factor = ref_M[i][c]
        for j in range(c, n + 1):
            ref_M[i][j] -= factor * ref_M[r][j]

    pivot_rows_cols.append((r, c))
    r += 1
    c += 1

# 3. Backward elimination to RREF
rref_M = clone_matrix(ref_M, m, n + 1)
num_pivots = custom_len(pivot_rows_cols)
for p_idx in range(num_pivots - 1, -1, -1):
    r_pivot, c_pivot = pivot_rows_cols[p_idx]
    for i in range(r_pivot - 1, -1, -1):
        factor = rref_M[i][c_pivot]
        for j in range(c_pivot, n + 1):
            rref_M[i][j] -= factor * rref_M[r_pivot][j]

return ref_M, rref_M, pivot_rows_cols

# Run the Gaussian reduction on the full augmented system
ref_result, rref_result, pivots = compute_ref_rref_pure(A, b, m, n)

print("\n" + "="*80)
print("                                GAUSSIAN ELIMINATION RESULTS                                ")
print("="*80)
print("\n--- Row Echelon Form (REF) of [A | b] ---")
for row in ref_result:
    print "[" + ", ".join(f"{x:12.8f}" for x in row) + "]"

print("\n--- Reduced Row Echelon Form (RREF) of [A | b] ---")
for row in rref_result:
    print "[" + ", ".join(f"{x:12.8f}" for x in row) + "]"

```

--- Interactive Matrix Configuration ---

Enter number of rows (m) for matrix A: 5

Enter number of columns (n) for matrix A (must be strictly greater than 5): 7

Enter the size of vector b (must equal m which is 5): 5

Successfully generated the random linear system!

--- Coefficient Matrix A ---

[8.38760492,	4.52137671,	9.73226419,	8.90912380,	6.35701124,	9.40776854,	5.10000000]
[6.50913825,	1.42278765,	2.59707773,	7.86901339,	4.46918288,	3.17738454,	6.00000000]
[9.99757804,	5.90887450,	6.90499391,	8.83531623,	3.93636413,	6.54742800,	2.00000000]
[7.12518361,	9.50636253,	2.84523254,	4.10655149,	5.83112536,	7.88887202,	5.00000000]
[8.77327090,	5.29544614,	6.12631720,	3.13305468,	9.73594097,	4.79954502,	6.00000000]

--- Vector b ---

[6.80147232]
[6.89715344]
[2.68005417]
[6.53974045]
[6.72818751]

=====

GAUSSIAN ELIMINATION RESULTS

=====
 --- Row Echelon Form (REF) of [A | b] ---

[	1.00000000,	0.59103060,	0.69066667,	0.88374566,	0.39373177,	0.65490141,	(
[	0.00000000,	1.00000000,	-0.39203610,	-0.41364155,	0.57141118,	0.60858946,	(
[	0.00000000,	0.00000000,	1.00000000,	0.34930344,	0.87669257,	1.10925739,	(
[	0.00000000,	0.00000000,	0.00000000,	1.00000000,	-1.32710050,	0.24609251,	(
[	0.00000000,	0.00000000,	0.00000000,	0.00000000,	1.00000000,	0.35295539,	(

--- Reduced Row Echelon Form (RREF) of [A | b] ---

[	1.00000000,	0.00000000,	0.00000000,	0.00000000,	0.00000000,	-1.03820822,	(
[	0.00000000,	1.00000000,	0.00000000,	0.00000000,	0.00000000,	0.91817022,	(
[	0.00000000,	0.00000000,	1.00000000,	0.00000000,	0.00000000,	0.55024680,	(
[	0.00000000,	0.00000000,	0.00000000,	1.00000000,	0.00000000,	0.71449977,	(
[	0.00000000,	0.00000000,	0.00000000,	0.00000000,	1.00000000,	0.35295539,	(

```
def solve_pure_system(m, n, rref_M, pivot_rows_cols):
    """
    Analyzes the RREF matrix of size m x (n+1) to classify pivot/free columns
    and solve for particular and fundamental homogeneous solutions.
    """
    # 1. Extract only the column indices from pivot_rows_cols
    pivot_cols = []
    for coord in pivot_rows_cols:
        pivot_cols.append(coord[1]) # coord is (row, col)

    # 2. Identify free (non-pivot) columns
    non_pivot_cols = []
    for col in range(n):
        is_pivot = False
        for p_col in pivot_cols:
            if col == p_col:
                is_pivot = True
                break
        if not is_pivot:
            non_pivot_cols.append(col)

    # 3. Formulate Particular Solution (xp)
    # Pivot variables are populated from the augmented column b'; free variables remain
    xp = []
    for _ in range(n):
        xp.append(0.0)
    for coord in pivot_rows_cols:
        r_pivot, c_pivot = coord
        xp[c_pivot] = rref_M[r_pivot][n] # Access index n (the augmented column b')

    # 4. Formulate Fundamental Solutions to Ax = 0 (Null Space Basis)
    # Assign 1.0 to one target free variable at a time, keeping others 0.0,
    # and perform backward substitution of RREF entries.
    hom_solutions = []
    for c_free in non_pivot_cols:
        v_h = []
        for _ in range(n):
            v_h.append(0.0)
        v_h[c_free] = 1.0
        for coord in pivot_rows_cols:
            r_pivot, c_pivot = coord
            v_h[c_pivot] = -rref_M[r_pivot][c_free]
        hom_solutions.append(v_h)

    return pivot_cols, non_pivot_cols, xp, hom_solutions

# Call the solver using Colab's preserved workspace variables
p_cols, np_cols, xp_vec, hom_basis = solve_pure_system(m, n, rref_result, pivots)
print("--")
```

```

print("=====")
print("                P_COLS & FREE VARIABLES                ")
print("=====")
print("="*80)
print(f"Pivot Columns: {p_cols}")
print(f"Non-pivot (Free) Columns: {np_cols}\n")

print("---- Particular Solution (xp) ----")
print "[" + ", ".join(f"{val:12.8f}" for val in xp_vec) + "]"

print("---- Null Space Basis (v_i) to solve Ax = 0 ----")
for idx, v in enumerate(hom_basis):
    print(f"  v_{idx+1}: [" + ", ".join(f"{val:12.8f}" for val in v) + "]")
print()

# Construct mathematical solution representation
terms = [f"c_{i+1} * v_{i+1}" for i in range(custom_len(hom_basis))]
print(f"General Solution Form: x = xp + " + " + ".join(terms))

```

```

=====
                P_COLS & FREE VARIABLES
=====
Pivot Columns: [0, 1, 2, 3, 4]
Non-pivot (Free) Columns: [5, 6]

--- Particular Solution (xp) ---
[ -1.20978293,   0.31704121,  -0.38953786,   1.09580789,   1.50126965,   0.00000000,   0.00000000]

--- Null Space Basis (v_i) to solve Ax = 0 ---
v_1: [  1.03820822,  -0.91817022,  -0.55024680,  -0.71449977,  -0.35295539,   1.00000000,   0.00000000]
v_2: [  0.82524949,  -0.07279275,   0.42796114,  -0.88268731,  -1.36769456,   0.00000000,   0.00000000]

General Solution Form: x = xp + c_1 * v_1 + c_2 * v_2

```

```

import random

# =====
# 1. PURE PYTHON MATHEMATICAL REPLACEMENTS (No Built-In Math/List Functions)
# =====

def custom_abs(x):
    """Returns the absolute value of x without the built-in abs()."""
    return x if x >= 0 else -x

def custom_len(lst):
    """Calculates the length of a list without the built-in len()."""
    count = 0
    for _ in lst:
        count += 1
    return count

def clone_matrix(M, rows, cols):
    """Deep clones a 2D matrix structure using nested loops (no slicing)."""
    cloned = []
    for i in range(rows):
        row = []
        for j in range(cols):
            row.append(M[i][j])
        cloned.append(row)
    return cloned

def generate_random_float():
    """
    Generates a random float of the form n.dddddddd (exactly 8 decimal places).
    Ensures compliance with Instruction 6 (strict 8-decimal entries).
    """
    integer_part = random.randint(1, 9)
    decimal_part = random.randint(10000000, 99999999)

```

```

    return float(f"{integer_part}.{decimal_part}")

def make_random_matrix(m, n):
    """Generates an m x n matrix populated with 8-decimal random floats."""
    M = []
    for _ in range(m):
        row = []
        for _ in range(n):
            row.append(generate_random_float())
        M.append(row)
    return M

def make_random_vector(m):
    """Generates an m x 1 vector represented as a flat list of 8-decimal floats."""
    v = []
    for _ in range(m):
        v.append(generate_random_float())
    return v

def matrix_vector_mult(A, x, rows, cols):
    """Computes matrix-vector multiplication Ax completely from scratch."""
    result = []
    for i in range(rows):
        total = 0.0
        for j in range(cols):
            total += A[i][j] * x[j]
        result.append(total)
    return result

# =====
# 2. CORE MATHEMATICAL SOLVER (GAUSSIAN ELIMINATION & BACK-SUBSTITUTION)
# =====

def compute_ref_rref_pure(A, b, m, n):
    """
    Constructs the augmented matrix [A | b] of size m x (n + 1) and computes
    its Row Echelon Form (REF) and Reduced Row Echelon Form (RREF) from scratch.
    """
    # Construct augmented matrix of size m x (n + 1)
    M = []
    for i in range(m):
        row = []
        for j in range(n):
            row.append(A[i][j])
        row.append(b[i]) # Append vector b entry at index n
        M.append(row)

    ref_M = clone_matrix(M, m, n + 1)
    r = 0
    c = 0
    pivot_rows_cols = [] # Stores pivot coordinate tuples (row, col)

    # Forward elimination loop to REF
    while r < m and c < n:
        pivot_row = r
        max_val = custom_abs(ref_M[r][c])
        for i in range(r + 1, m):
            val_abs = custom_abs(ref_M[i][c])
            if val_abs > max_val:
                max_val = val_abs
                pivot_row = i

        # Division-by-zero protection:
        if max_val < 1e-9:
            # Skip this column if all entries are zero (marks a free variable)
            c += 1

```

```

        continue

    # Row swap if a larger absolute entry was found below
    if pivot_row != r:
        temp = ref_M[r]
        ref_M[r] = ref_M[pivot_row]
        ref_M[pivot_row] = temp

    # Normalize the pivot row so the leading entry equals 1.0
    pivot_val = ref_M[r][c]
    for j in range(c, n + 1):
        ref_M[r][j] /= pivot_val

    # Eliminate entries in current column for all rows below
    for i in range(r + 1, m):
        factor = ref_M[i][c]
        for j in range(c, n + 1):
            ref_M[i][j] -= factor * ref_M[r][j]

    pivot_rows_cols.append((r, c))
    r += 1
    c += 1

# Backward elimination loop to RREF
rref_M = clone_matrix(ref_M, m, n + 1)
num_pivots = custom_len(pivot_rows_cols)
for p_idx in range(num_pivots - 1, -1, -1):
    r_pivot, c_pivot = pivot_rows_cols[p_idx]
    for i in range(r_pivot - 1, -1, -1):
        factor = rref_M[i][c_pivot]
        for j in range(c_pivot, n + 1):
            rref_M[i][j] -= factor * rref_M[r_pivot][j]

return ref_M, rref_M, pivot_rows_cols

def solve_pure_system(m, n, rref_M, pivot_rows_cols):
    """
    Analyzes the RREF matrix of size m x (n+1) to classify pivot/free columns
    and solve for particular and fundamental homogeneous solutions.
    """
    # 1. Extract only the column indices of the pivots
    pivot_cols = []
    for coord in pivot_rows_cols:
        pivot_cols.append(coord[1])

    # 2. Identify free (non-pivot) columns
    non_pivot_cols = []
    for col in range(n):
        is_pivot = False
        for p_col in pivot_cols:
            if col == p_col:
                is_pivot = True
                break
        if not is_pivot:
            non_pivot_cols.append(col)

    # 3. Formulate Particular Solution (xp)
    # Pivot variables are populated from the augmented column b'; free variables remain
    xp = []
    for _ in range(n):
        xp.append(0.0)
    for coord in pivot_rows_cols:
        r_pivot, c_pivot = coord
        xp[c_pivot] = rref_M[r_pivot][n] # Access index n (the augmented column b')

    # 4. Formulate Fundamental Solutions to Ax = 0 (Null Space Basis)
    # Assign 1.0 to one target free variable at a time, keeping others 0.0

```

```

# Assign 1 to one target free variable at a time, keeping others 0,
# and perform backward substitution of RREF entries.
hom_solutions = []
for c_free in non_pivot_cols:
    v_h = []
    for _ in range(n):
        v_h.append(0.0)
    v_h[c_free] = 1.0
    for coord in pivot_rows_cols:
        r_pivot, c_pivot = coord
        v_h[c_pivot] = -rref_M[r_pivot][c_free]
    hom_solutions.append(v_h)

    return pivot_cols, non_pivot_cols, xp, hom_solutions

# =====
# 3. EXECUTION FLOW
# =====

def run_experiment():
    # Set seed for reproducible assignment results
    random.seed(54321)

    m = 5
    n = 7

    print("=" * 80)
    print("                QUESTION 1: LINEAR ALGEBRA SYSTEM SOLVER (PURE PYTHON)                ")
    print("=" * 80 + "\n")

    # Generate A and b
    A = make_random_matrix(m, n)
    b = make_random_vector(m)

    print("---- Coefficient Matrix A (5x7) ----")
    for row in A:
        print "[" + ", ".join("{x:12.8f}" for x in row) + "]"
    print()

    print("---- Right-Hand Side Vector b (5x1) ----")
    for val in b:
        print(f"[{val:12.8f}]")
    print()

    # Compute Row Reduction on augmented matrix [A | b]
    ref_M, rref_M, pivot_coords = compute_ref_rref_pure(A, b, m, n)

    print("---- Row Echelon Form (REF) of [A | b] ----")
    for row in ref_M:
        print "[" + ", ".join("{x:12.8f}" for x in row) + "]"
    print()

    print("---- Reduced Row Echelon Form (RREF) of [A | b] ----")
    for row in rref_M:
        print "[" + ", ".join("{x:12.8f}" for x in row) + "]"
    print()

    # Solve System
    p_cols, np_cols, xp, hom_basis = solve_pure_system(m, n, rref_M, pivot_coords)

    print(f"Pivot Columns: {p_cols}")
    print(f"Non-pivot Columns (Free Variables): {np_cols}\n")

    print("Particular Solution (xp):")
    print "[" + ", ".join("{val:12.8f}" for val in xp) + "]\n")

    print("Solutions to Ax = 0 (Null Space Basis):")

```

```

for idx, v in enumerate(hom_basis):
    print(f" v_{idx+1}: [" + ", ".join(f"{val:12.8f}" for val in v) + "]")
print()

# Formulate mathematical expressions
terms = [f"c_{i+1} * v_{i+1}" for i in range(custom_len(hom_basis))]
print(f"General Solution Form: x = xp + " + " + ".join(terms) + "\n")

# =====
# 4. SOLUTIONS VERIFICATION
# =====

print("=" * 80)
print("                                VERIFICATION SUITE                                ")
print("=" * 80 + "\n")

# A. Particular solution check (Ax_p = b)
ax_p = matrix_vector_mult(A, xp, m, n)
p_err_sq = 0.0
for i in range(m):
    diff = ax_p[i] - b[i]
    p_err_sq += diff * diff
p_err = p_err_sq ** 0.5
print("A * xp:")
print([" + ", ".join(f"{val:12.8f}" for val in ax_p) + "]")
print("Vector b:")
print([" + ", ".join(f"{val:12.8f}" for val in b) + "]")
print(f"--> Particular Solution L2 Residual Error: {p_err:.12e}\n")

# B. Homogeneous solution check (A * v_i = 0)
for idx, v in enumerate(hom_basis):
    ax_h = matrix_vector_mult(A, v, m, n)
    h_err_sq = 0.0
    for i in range(m):
        h_err_sq += ax_h[i] * ax_h[i]
    h_err = h_err_sq ** 0.5
    print(f"A * v_{idx+1}:")
    print([" + ", ".join(f"{val:12.8f}" for val in ax_h) + "]")
    print(f"--> Basis Vector v_{idx+1} L2 Residual Error: {h_err:.12e}\n")

# C. General Solution check with arbitrary constants c1 and c2
c1 = generate_random_float()
c2 = generate_random_float()
print(f"Selected Arbitrary Constants: c_1 = {c1:.8f}, c_2 = {c2:.8f}")

x_gen = []
for j in range(n):
    coord_val = xp[j]
    if custom_len(hom_basis) >= 1:
        coord_val += c1 * hom_basis[0][j]
    if custom_len(hom_basis) >= 2:
        coord_val += c2 * hom_basis[1][j]
    x_gen.append(coord_val)

ax_gen = matrix_vector_mult(A, x_gen, m, n)
gen_err_sq = 0.0
for i in range(m):
    diff = ax_gen[i] - b[i]
    gen_err_sq += diff * diff
gen_err = gen_err_sq ** 0.5

print("\nGeneral Solution Vector x_gen:")
print([" + ", ".join(f"{val:12.8f}" for val in x_gen) + "]")
print("A * x_gen:")
print([" + ", ".join(f"{val:12.8f}" for val in ax_gen) + "]")
print(f"--> General Solution L2 Residual Error: {gen_err:.12e}")
print("=" * 80)

```

```
if __name__ == "__main__":
    run_experiment()
```

QUESTION 1: LINEAR ALGEBRA SYSTEM SOLVER (PURE PYTHON)

-- Coefficient Matrix A (5x7) ---

```
8.19315408, 4.32912687, 9.80928415, 7.33518243, 6.79060523, 5.26243125, 1
8.13173622, 2.98845720, 2.79240004, 4.37833697, 2.14642614, 8.60044066, 9
6.61920071, 4.15568598, 8.20771206, 1.34611603, 7.61020507, 7.52892473, 6
9.22297126, 3.49185192, 1.99968705, 6.73476967, 4.89663855, 3.54655579, 1
1.19424970, 5.36056138, 5.74031902, 7.26616743, 3.51744516, 1.44465294, 6
```

-- Right-Hand Side Vector b (5x1) ---

```
7.30816572]
5.32988719]
1.21240731]
6.89280129]
6.51325439]
```

-- Row Echelon Form (REF) of [A | b] ---

```
1.00000000, 0.37860379, 0.21681593, 0.73021692, 0.53091768, 0.38453506, 0
0.00000000, 1.00000000, 1.11673277, 1.30268274, 0.58743963, 0.20076181, 1
0.00000000, 0.00000000, 1.00000000, -0.03695443, 0.25813709, 0.28000288, -0
0.00000000, 0.00000000, 0.00000000, 1.00000000, -0.33996269, -0.59990097, -0
0.00000000, 0.00000000, 0.00000000, 0.00000000, 1.00000000, -1.50234729, -2
```

-- Reduced Row Echelon Form (RREF) of [A | b] ---

```
1.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000, 1.16436117, 0
0.00000000, 1.00000000, 0.00000000, 0.00000000, 0.00000000, 1.83017960, 4
0.00000000, 0.00000000, 1.00000000, 0.00000000, 0.00000000, 0.62677126, 0
0.00000000, 0.00000000, 0.00000000, 1.00000000, 0.00000000, -1.11064299, -1
0.00000000, 0.00000000, 0.00000000, 0.00000000, 1.00000000, -1.50234729, -2
```

ivot Columns: [0, 1, 2, 3, 4]

in-pivot Columns (Free Variables): [5, 6]

rticular Solution (xp):

```
0.19848946, 0.04487188, 0.07161624, 0.90748548, -0.27558968, 0.00000000, 0
```

olutions to $Ax = 0$ (Null Space Basis):

```
v_1: [-1.16436117, -1.83017960, -0.62677126, 1.11064299, 1.50234729, 1.00000000]
v_2: [-0.95676844, -4.69939799, -0.45882485, 1.67953758, 2.78038945, 0.00000000]
```

eneral Solution Form: $x = x_p + c_1 * v_1 + c_2 * v_2$

VERIFICATION SUITE

* xp:

```
7.30816572, 5.32988719, 1.21240731, 6.89280129, 6.51325439]
```

ector b:

```
7.30816572, 5.32988719, 1.21240731, 6.89280129, 6.51325439]
```

-> Particular Solution L2 Residual Error: 1.110223024625e-15

* v_1:

```
-0.00000000, -0.00000000, -0.00000000, -0.00000000, 0.00000000]
```

-> Basis Vector v_1 L2 Residual Error: 8.837263695108e-15

